

Practical: 5

Implementation of a knapsack problem using dynamic programming

Aim: Implementation of a knapsack problem using dynamic programming

ALGORITHM:

Input :- n = No. of items
 W = knapsack capacity
 $W[i]$ = weight of item i
 $V[i]$ = value of item i
Output :- Maximum possible value
Selected item.

Step 1:- Start
Step 2:- Read the no. of items n & knapsack capacity W
Step 3:- Read the weight $W[i]$ & value $V[i]$ of every item
Step 4:- Create a dp table $dp[n+1][W+1]$
Step 5:- Initialize the 1st row & 1st column of the dp table to 0
Step 6:- For every item i from 1 to n ,
• for every capacity j from 1 to W .
Step 7:- Check whether the weight of the current item is less than or equal to the current capacity.
• If $W[i] \leq j$, calculate two possibilities:
• Take the item: $V[i] + dp[i-1][j-W[i]]$
• Do not take the item: $dp[i-1][j]$
Step 8:- Store the max of these two values:
 $dp[i][j] = \max(V[i] + dp[i-1][j-W[i]], dp[i-1][j])$
Step 9:- If $W[i] > j$, the item can not be included:
 $dp[i][j] = dp[i-1][j]$
Step 10:- After filling the table, $dp[n][W]$ contains the max val.
Step 11:- Track the dp table from $dp[n][W]$ back wards to determine items were selected.
Step 12:- Display the max value & selected items.
Step 13:- Stop.

PROGRAM

```
#include<iostream>
using namespace std;

class Knapsack {

private:
    int w[10];    // weights
    int v[10];    // values
    int n;        // number of items
    int W;        // capacity
    int dp[10][100]; // dp table

public:

    void getInput() {
        cout << "Enter number of items  : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin >> w[i] >> v[i];
        }
    }

    void solve() {

        // base case — row 0 and col 0
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;

        // fill table
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= W; j++) {

                if(w[i] <= j) {
                    // take or not take
                    dp[i][j] = max(
                        v[i] + dp[i-1][j-w[i]],
```

```
        dp[i-1][j]
    );
} else {
    // cannot take item i
    dp[i][j] = dp[i-1][j];
}
}
}
}

void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "    ";
    for(int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;

    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;
    }
}

void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken " << endl;
        }
    }
}
```

```
}  
  
void showResult() {  
    cout << "\nMaximum Value = "  
        << dp[n][W] << endl;  
}  
};  
  
int main() {  
  
    Knapsack k;  
  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
  
    return 0;  
}
```

OUTPUT:

```
Output
Enter number of items : 4
Enter knapsack capacity : 8
Item 1 weight and value : 2
1
Item 2 weight and value : 3
2
Item 3 weight and value : 4
5
Item 4 weight and value : 5
6

DP Table:
  0  1  2  3  4  5  6  7  8
i=0 0  0  0  0  0  0  0  0  0
i=1 0  0  1  1  1  1  1  1  1
i=2 0  0  1  2  2  3  3  3  3
i=3 0  0  1  2  5  5  6  7  7
i=4 0  0  1  2  5  6  6  7  8

Maximum Value = 8

Selected Items:
Item 4 (w=5, v=6) TAKEN
Item 3 (w=4, v=5) NOT taken
Item 2 (w=3, v=2) TAKEN
Item 1 (w=2, v=1) NOT taken

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time complexity

let n = No. of items
 W = Knapsack capacity
Solve() function

The program contains two nested loops.

```
for (int i = 1; i < n; i++) {  
    for (int j = 1; j <= w; j++) {
```

- The outer loop executes n times
- The inner loop executes w times for every item.

Therefore $T(n, W) = n \times W$

Time complexity $\therefore T(n) = O(nW)$

Best case! even if the weights are large & many items cannot be selected, the program still fills the table $O(nW)$

Average case! The program checks all items & all capacity $T(n) = O(nW)$

Worst case! The program checks all items & all capacity $T(n) = O(nW)$

CONCLUSION:

The 0/1 Knapsack Problem was successfully implemented using Dynamic Programming. The approach efficiently finds the maximum value that can be obtained within the given weight capacity by breaking the problem into smaller subproblems and storing their results. This avoids repeated calculations and improves efficiency compared with a simple recursive approach. Thus, Dynamic Programming provides an effective and systematic solution for solving the Knapsack Problem.